



Write-up: CHALL

por *dleskow*

Informações do chall

- **Título:** CTF/CHALL
 - **Autor:** ??
 - **Contexto:** Basta inserir aqui
- **Arquivos:**
 - *chall.zip*
 - *chall* (ELF)
 - *libc.so.6*
 - *ld-linux-x86-64.so.2*
 - *Dockerfile*
 - *run.sh*
 - *flag.txt*

1. Write-Up

1.1. Vulnerabilidade

```
(dleskow@ganesh) - [/home/ganesh/CTF/CHALL]
# Ctrl+C, Ctrl+V terminal commands and/or program execution like this
# Or if you want the xxd of something? idk
(dleskow@ganesh) - [/home/ganesh/CTF/CHALL]
# xxd file
00000000: 4374 726c 2b43 2c20 4374 726c 2b56 2074 Ctrl+C, Ctrl+V t
00000010: 6572 6d69 6e61 6c20 636f 6d6d 616e 6473 erminal commands
00000020: 206c 696b 6520 7468 6973 0a          like this.
```

```
pwndbg> Ctrl+C, Ctrl+V pwndbg output like this:
pwndbg> info proc m
0x00007f23b8b99000 0x00007f23b8b9b000 0x2000 0x1e6000 rw-p /home/ganesh/CTF/CHALL/libc.so.6

pwndbg> hexdump 0x00007f23e7c0000 0x5000
+16e0 0x7f23b8b9a6e0 08 f0 f0 6c fc 7f 00 00 00 00 00 00 00 00 |...l...|.....|

pwndbg> b* main+208
Breakpoint 2 at 0x562cb46585a1
pwndbg> c
Continuing.

pwndbg> p $rbp
$2 = (void *) 0x7ffc6cf0eef0
```

```
unsigned __int64 __fastcall show_ida_functions(unsigned __int8 a1)
{
    canary = __readfsqword(0x28u);
    printf("Ctrl+C, Ctrl+V IDA functions here: %p\n", offset);
    return canary - __readfsqword(0x28u);
}
```

Ataque utilizando *pwntools*

Ataque

Vazamento de endereços ou sei lá

Descrição



```
def template():  
    print('Ctrl+C, Ctrl+V python code like this')  
if 2+2 == 5:  
    p_equals_np_proof()  
else:  
    template()
```

Etc

2. solve.py

```
from pwn import *  
  
exe = ELF("./notes_patched")  
libc = ELF("./libc.so.6", checksec=False)  
ld = ELF("./ld-linux-x86-64.so.2", checksec=False)  
  
REMOTE = False  
REMOTE = True  
  
if REMOTE:  
    context.bits = 64  
    p = remote('ganesh.icmc.usp.br', 5005)  
    #p = remote('localhost', 5005)  
else:  
    context.binary = exe  
    context.terminal = [ 'qterminal', '-e' ]  
    p = process([exe.path])  
    #gdb.attach(p, exe=exe.path, gdbscript="b* main+413\ncontinue")  
    gdb.attach(p, exe=exe.path, gdbscript="continue")  
  
#context.log_level = 'debug'  
  
# Bla  
# Bla  
# Bla  
  
p.interactive()
```